



Artificial Neural Network Lab (ANN) LAB 08

Lab Demonstrator: Shakeela Shaheen

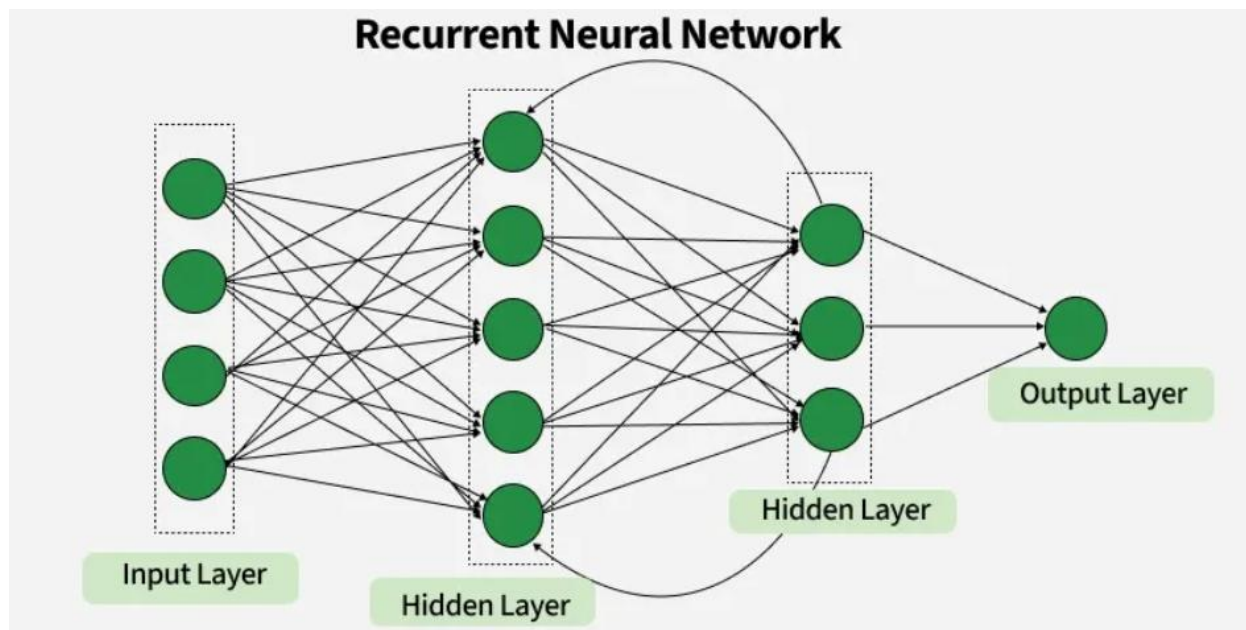


LAB 08: IMPLEMENTING RECURRENT NEURAL NETWORKS (RNN)

Recurrent Neural Networks

Recurrent Neural Networks (RNNs) are a class of neural networks designed to process sequential data by retaining information from previous steps. They are especially effective for tasks where context and order matter.

- Designed for sequential and temporal data
- Maintains memory of past inputs
- Widely used in NLP, forecasting and speech tasks



Lets understand RNN with a example:

Imagine reading a sentence and you try to predict the next word, you don't rely only on the current word but also remember the words that came before. RNNs work similarly by "remembering" past information i.e it considers all the earlier words to choose the most likely next word.

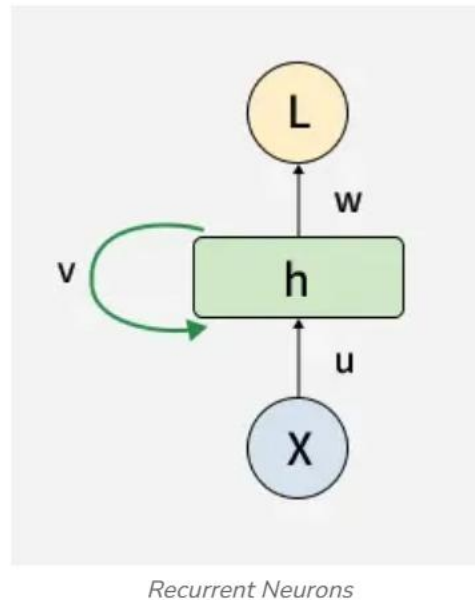
This memory of previous steps helps the network understand context and make better predictions.

Key Components of RNNs

There are mainly two components of RNNs that we will discuss.

1. Recurrent Neurons

The fundamental processing unit in RNN is a Recurrent Unit. They hold a hidden state that maintains information about previous inputs in a sequence. Recurrent units can "remember" information from prior steps by feeding back their hidden state, allowing them to capture dependencies across time.



We have 3 review sentences:

Review No.	Sentence	Sentiment
1	movie was good	1
2	movie was bad	0
3	movie was not good	0

Sentiment Labels

- 1 = Positive
- 0 = Negative

Step 1: Vocabulary

From the sentences, the unique words are:

- movie
- was
- good
- bad
- not

So the **vocabulary size** = 5

Step 2: One-Hot Encoding

Each word is converted into a vector of length 5.

One-hot vectors

Word	One-Hot Vector
movie	[1 0 0 0 0]
was	[0 1 0 0 0]
good	[0 0 1 0 0]
bad	[0 0 0 1 0]
not	[0 0 0 0 1]

Step 3: Input Sentences as Sequences

Review 1: movie was good

- (x_1 =) movie = [1 0 0 0 0]
- (x_2 =) was = [0 1 0 0 0]
- (x_3 =) good = [0 0 1 0 0]

Review 2: movie was bad

- (x_1 =) movie = [1 0 0 0 0]
- (x_2 =) was = [0 1 0 0 0]
- (x_3 =) bad = [0 0 0 1 0]

Review 3: movie was not good

- (x_1 =) movie = [1 0 0 0 0]
- (x_2 =) was = [0 1 0 0 0]
- (x_3 =) not = [0 0 0 0 1]
- (x_4 =) good = [0 0 1 0 0]

Why RNN is Needed Here

Look at these two sentences:

1. **movie was good** → Positive
2. **movie was not good** → Negative

A simple model may see the word **good** and think both are positive.

But RNN can understand sequence:

- **not + good** changes the meaning.

So RNN learns **context**.

RNN Working Step by Step

At time step ($t = 1$)

Input word = **movie**

```
[  
x_1 = [1\ 0\ 0\ 0\ 0]  
]
```

This goes into the RNN cell.

The RNN computes hidden state:

```
[  
h_1 = f(W_x x_1 + W_h h_0 + b)  
]
```

Since this is the first word:

```
[  
h_0 = 0  
]
```

So:

```
[  
h_1 = f(W_x x_1 + b)  
]
```

At time step ($t = 2$)

Input word = **was**

```
[  
x_2 = [0\ 1\ 0\ 0\ 0]  
]
```

Now the RNN uses:

- current input (x_2)
- previous memory (h_1)

```
[  
h_2 = f(W_x x_2 + W_h h_1 + b)  
]
```

So the network now remembers:

- previous word = **movie**
- current word = **was**

At time step ($t = 3$)

Input word = **good** / **bad** / **not**

Example for **good**:

[
x₃ = [0\ 0\ 1\ 0\ 0]
]

Then:

[
h₃ = f(W_x x₃ + W_h h₂ + b)
]

Now the hidden state contains information from the whole sentence.

Final Output

After reading the full sentence, the RNN gives output:

[
y = \sigma(W_y h_t + b_y)
]

Where:

- ($y \approx 1$) → Positive
- ($y \approx 0$) → Negative

1) Input Sequence

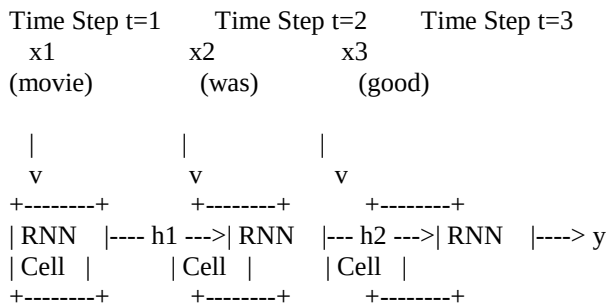
Sentence: "movie was good"

x1	x2	x3
movie -->	was -->	good

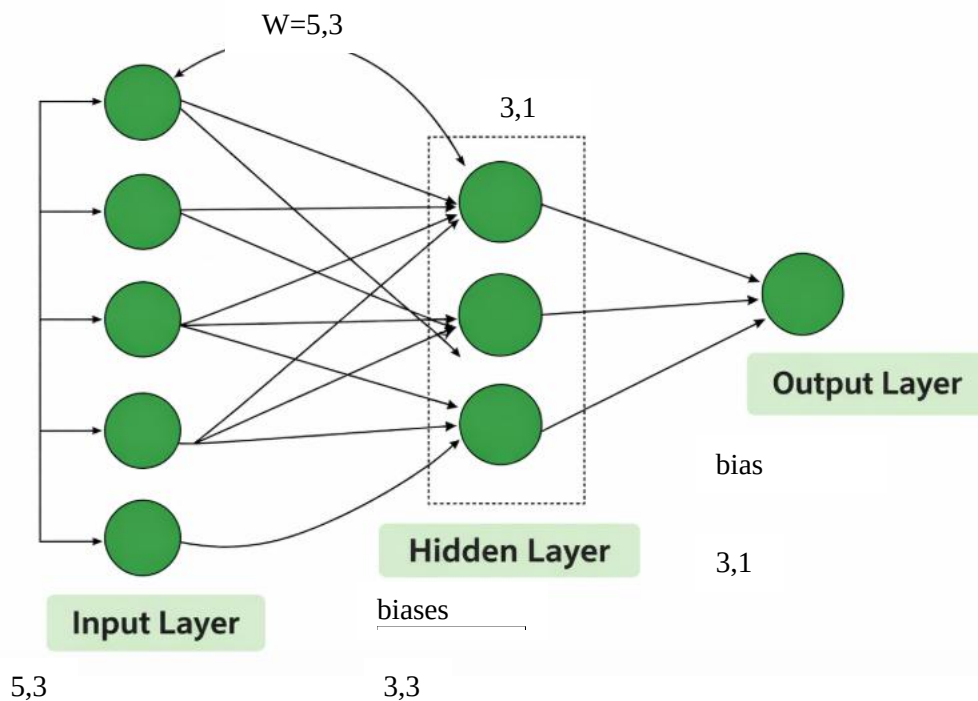
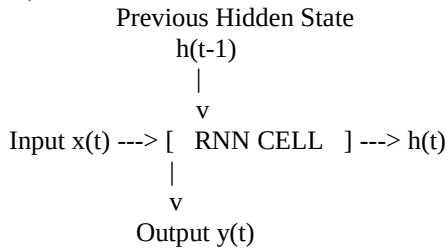
2) One-Hot Encoded Inputs

movie = [1 0 0 0 0]
was = [0 1 0 0 0]
good = [0 0 1 0 0]
bad = [0 0 0 1 0]
not = [0 0 0 0 1]

3) RNN Unrolled Diagram



4) Internal Structure of One RNN Cell

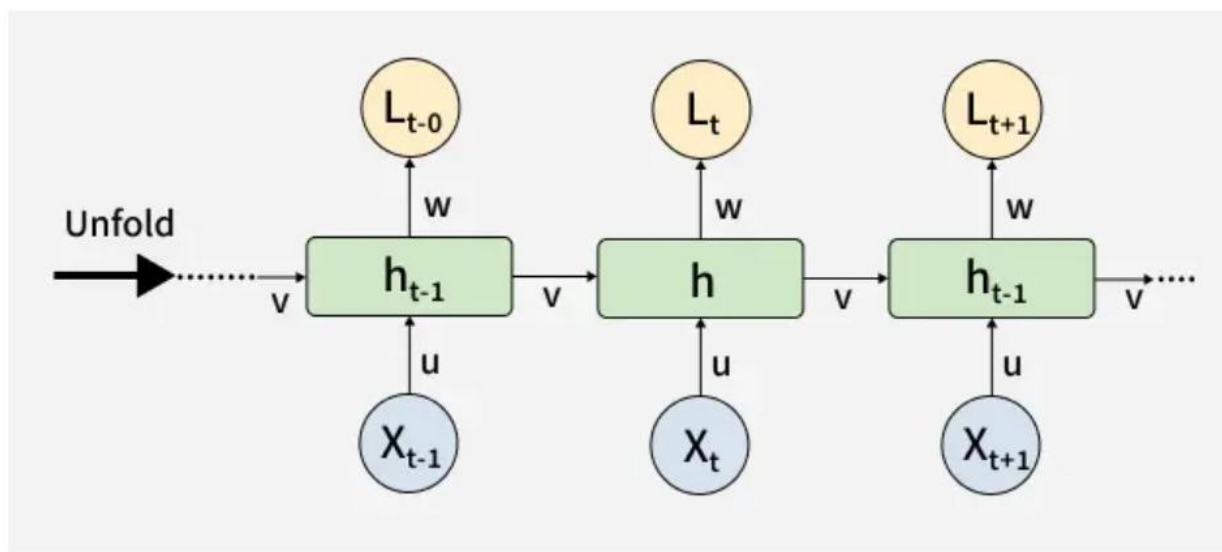


2. RNN Unfolding

RNN unfolding or unrolling is the process of expanding the recurrent structure over time steps. During unfolding each step of the sequence is represented as a separate layer in a series illustrating how information flows across each time step.

This unrolling enables [backpropagation through time \(BPTT\)](#) a learning process where errors are propagated across time steps to adjust the network's weights enhancing the RNN's ability to

learn dependencies within sequential data.



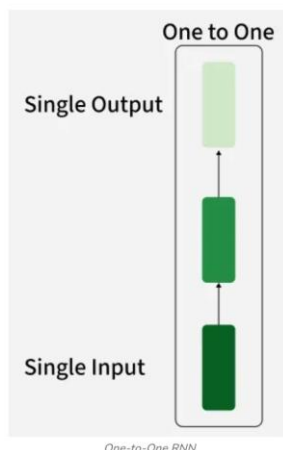
RNN Unfolding

Types Of Recurrent Neural Networks

There are four types of RNNs based on the number of inputs and outputs in the network:

1. One-to-One RNN

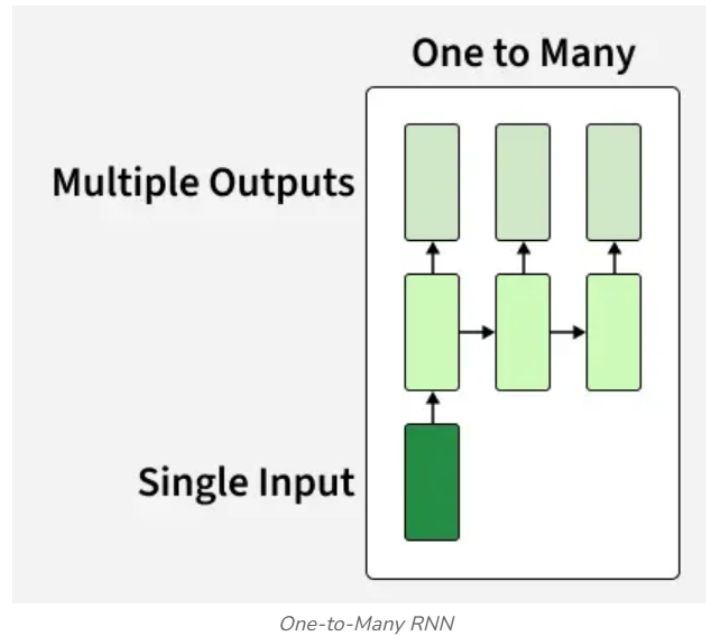
This is the simplest type of neural network architecture where there is a single input and a single output. It is used for straightforward classification tasks such as binary classification where no sequential data is involved.



One-to-One RNN

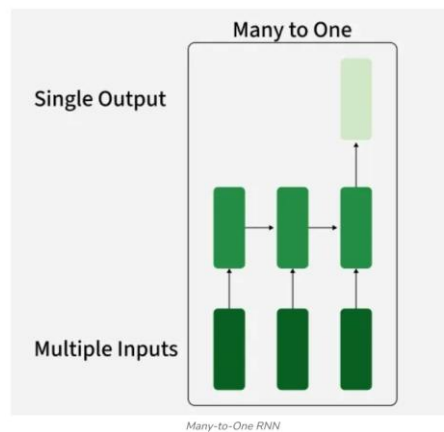
2. One-to-Many RNN

In a One-to-Many RNN the network processes a single input to produce multiple outputs over time. This is useful in tasks where one input triggers a sequence of predictions (outputs). For example in image captioning a single image can be used as input to generate a sequence of words as a caption.



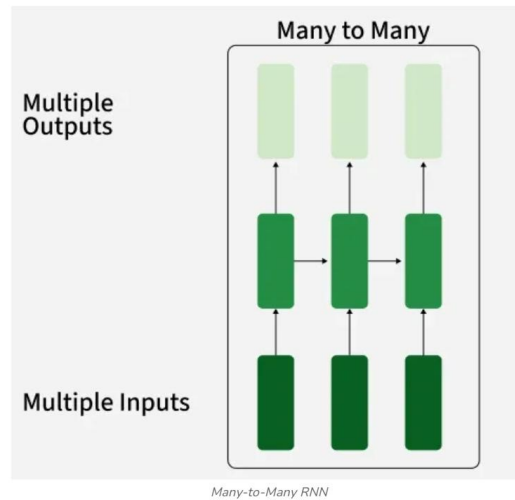
3. Many-to-One RNN

The Many-to-One RNN receives a sequence of inputs and generates a single output. This type is useful when the overall context of the input sequence is needed to make one prediction. In sentiment analysis the model receives a sequence of words (like a sentence) and produces a single output like positive, negative or neutral.



4. Many-to-Many RNN

The Many-to-Many RNN type processes a sequence of inputs and generates a sequence of outputs. In language translation task a sequence of words in one language is given as input and a corresponding sequence in another language is generated as output.



Implementing a Text Generator Using Recurrent Neural Networks (RNNs)

In this section, we create a character-based text generator using Recurrent Neural Network (RNN) in TensorFlow and Keras. We'll implement an RNN that learns patterns from a text sequence to generate new text character-by-character.

1. Importing Necessary Libraries

We start by importing essential libraries for data handling and building the neural network.

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense
```

2.

Defining the Input Text and Prepare Character Set

We define the input text and identify unique characters in the text which we'll encode for our model.

```
text = "This is GeeksforGeeks a software training institute"
chars = sorted(list(set(text)))
char_to_index = {char: i for i, char in enumerate(chars)}
index_to_char = {i: char for i, char in enumerate(chars)}
```

3.

Creating Sequences and Labels

To train the RNN, we need sequences of fixed length (`seq_length`) and the character following each sequence as the label.

```
seq_length = 3
sequences = []
labels = []

for i in range(len(text) - seq_length):
    seq = text[i:i + seq_length]
    label = text[i + seq_length]
    sequences.append([char_to_index[char] for char in seq])
    labels.append(char_to_index[label])

X = np.array(sequences)
y = np.array(labels)
```

4. Converting Sequences and Labels to One-Hot Encoding

For training we convert `x` and `y` into one-hot encoded tensors.

```
X_one_hot = tf.one_hot(X, len(chars))
y_one_hot = tf.one_hot(y, len(chars))
```

5. Building the RNN Model

We create a simple RNN model with a hidden layer of 50 units and a Dense output layer with [softmax activation](#).

```
model = Sequential()
model.add(SimpleRNN(50, input_shape=(seq_length, len(chars)),
    activation='relu'))
model.add(Dense(len(chars), activation='softmax'))
```

6. Compiling and Training the Model

We compile the model using the `categorical_crossentropy` loss and train it for 100 epochs.

```
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
model.fit(X_one_hot, y_one_hot, epochs=100)
```

Output:

```
2/2 ————— 0s 29ms/step - accuracy: 0.9618 - loss: 0.0664
Epoch 94/100
2/2 ————— 0s 26ms/step - accuracy: 0.9514 - loss: 0.0729
Epoch 95/100
2/2 ————— 0s 26ms/step - accuracy: 0.9514 - loss: 0.0661
Epoch 96/100
2/2 ————— 0s 26ms/step - accuracy: 0.9514 - loss: 0.0655
Epoch 97/100
2/2 ————— 0s 28ms/step - accuracy: 0.9514 - loss: 0.0652
Epoch 98/100
2/2 ————— 0s 26ms/step - accuracy: 0.9618 - loss: 0.0648
Epoch 99/100
2/2 ————— 0s 29ms/step - accuracy: 0.9618 - loss: 0.0590
Epoch 100/100
2/2 ————— 0s 26ms/step - accuracy: 0.9618 - loss: 0.0583
<keras.src.callbacks.history.History at 0x7a5e40a739d0>
```

Training the RNN model

Lab Task

1.Dataset

- **Text Classification** (e.g., Sentiment Analysis)

2. Model Development

- Build an RNN model using:
 - **SimpleRNN**
- You may use any of the following frameworks:
 - TensorFlow / Keras
 - PyTorch

3. Dataset Requirements

- Use a **publicly available dataset** (e.g., IMDb, Twitter sentiment dataset)

Clearly mention the following in your report:

- Dataset source
- Input and output format
- Preprocessing steps (e.g., tokenization, padding, encoding)

4. Model Training

- Split the dataset into:
 - Training set
 - Testing set
- Train the model and record:
 - **Loss vs Epochs**
 - **Accuracy vs Epochs** (for classification tasks)

5. Visualization

Plot the following graphs:

- Training Loss vs Validation Loss
- Accuracy vs Epochs

Also include:

- Sample predictions from the model

6. Discussion (5–7 Lines)

Write a short explanation covering:

- How the RNN works
- Why SimpleRNN is suitable for your task
- Performance evaluation of your model